Code storylines: semantic maps, Etchpad and a reproducible prototype

Researched 2026-10-12.

What is actually shown

Muse describes a TypeScript inventory example laid out as entry functions -> Inventory -> Product/Order/OrderItem/OrderStatus. This is a dependency explanation, not proof that all real execution paths can be statically recovered.

Current resource

Etchpad (https://etchpad.ai) currently describes scanning a local codebase into a procedural semantic map, with a separate optional AI walkthrough. Those are vendor claims; no independent speedup benchmark was established in this check. Use its own access/download flow. A public package install command was not established, so none is invented here.

Reproduce the idea without depending on access

1. Choose one small TypeScript module with known entry points and tests.
2. Extract declarations and references using the compiler's syntax/type facilities. Keep symbol IDs, source file ranges and unresolved calls.
3. Distinguish call edges, type references, inheritance and imports. Label them separately; a type dependency is not an execution step.
4. Lay out entry points on the left, called behavior in the middle and supporting types on the right. Collapse cycles into groups rather than claiming a unique ordering.
5. Clicking a block should open the source range; changing code should invalidate the corresponding map.
6. Compare the map with a debugger trace on two test cases. Display dynamic/unresolved dispatch explicitly.

Limits to test

Reflection, dynamic imports, generated code, overloaded methods and configuration-dependent paths can defeat a static map. "Deterministically generated" does not mean the map has proven every runtime fact. Keep AI explanations visually distinct from parsed references. The papers named by Muse remain suggested reading, not evidence of a measured productivity gain for this product.

Project acceptance

A useful prototype supports one language, handles a cyclic dependency, exposes unresolved edges and survives a source edit. Ask another developer to explain the same module using the file and then the map;

record task accuracy and time without assuming the map wins. Export a PNG/SVG only from trusted renderer code, with no embedded execution from a source comment.

Reusable project prompt

```
Build Source-to-storyline explorer. Read-only semantic graph for one TypeScript project, with source navigation and uncertainty labels. Read the linked official docs first. Use fixture data; keep credentials outside source control. Deliver tests, setup instructions, an evidence/limitations note and a small reproducible example. Do not claim a measurement you did not run. Milestones: Extract symbols and references; Render typed edges and cycle groups; Add source navigation and stale-map detection; Compare against debugger traces.
```

Delivery and next step

The linked current sources were retrieved for this guide; the original dossier and its extraction caveats are retained in Sources. The guide and printable PDF are research deliverables. The proposed project has not been built. Start with the first milestone and verify its acceptance criteria before expanding the scope.

Original sources

- https://www.instagram.com/reel/DeKWvd1BAX-/